Empirical Evaluation of Random Forest Ensembles for Multi-Class Network Intrusion Detection

Student Number: 26895064
Computer Science Division, Stellenbosch University
Email: 26895064@sun.ac.za
Code Repository: https://github.com/swantaan/ML_ass4

***Abstract*—Network intrusion detection systems require robust classification models capable of identifying complex cyberattacks across severe class imbalances and heterogeneous feature spaces. This study investigates ensemble learning through Breiman's random forest algorithm, comparing predictive capability against classification and regression trees and k-nearest neighbours on the UNSW-NB15 network traffic dataset. All models, including vectorized quantile split searching, bootstrap aggregation, and out-of-bag error estimation, were implemented from scratch in pure NumPy without third-party machine learning libraries. An empirical evaluation was conducted across 257 673 network connection records partitioned into an 80% stratified training set and a 20% test set using random seed 69. The random forest ensemble achieved an overall test accuracy of 81.83% and a macro precision of 84.52%, outperforming the single decision tree baseline of 80.97% accuracy and 72.63% macro precision, and the k-nearest neighbours baseline of 65.13% accuracy. Out-of-bag error estimates converged asymptotically to the test generalization error within a margin of 0.20%, establishing out-of-bag validation as an efficient alternative to cross-validation. An ablation study demonstrated that random subspace feature sampling of $m = \lfloor \sqrt{D} \rfloor$ features per split reduced training wall-clock time by 77.5% relative to full bagging while preserving classification accuracy. Mean decrease in impurity identified destination service connection counts and source time-to-live values as the primary discriminative features.**

***Index Terms*—Network intrusion detection, random forest, bootstrap aggregation, random subspace method, out-of-bag estimation, decision trees, nearest neighbours.**

I. Introduction

Network security infrastructure depends on automated intrusion detection systems to distinguish benign communications from malicious activities. Modern transmission control protocol and internet protocol traffic generates high data throughput characterised by extreme class imbalance, multicollinearity, and heterogeneous descriptive features. Machine learning classifiers provide automated mechanisms to detect synthetic and contemporary attack vectors across network connection records.

Individual classification models exhibit distinct theoretical limitations when applied to high-dimensional network flow data. Instance-based learners such as k-nearest neighbours suffer from substantial computational overhead during inference and exhibit vulnerability to irrelevant descriptive features. Single decision tree models generate intuitive decision boundaries through recursive binary partitioning. However, single decision trees suffer from high variance and error propagation from sub-optimal splits near the tree root.

Ensemble learning addresses these individual vulnerabilities by combining predictions from multiple base models. Bootstrap aggregation reduces estimator variance by training individual trees on independent bootstrap samples. The random subspace method further decorrelates base learners by restricting split evaluations to a random subset of descriptive features. Combining bootstrap aggregation and feature subspace sampling yields the random forest architecture introduced by Breiman.

This investigation evaluates four core research questions:

1) Does a random forest ensemble achieve superior multi-class classification performance compared to a single classification tree and a k-nearest neighbours baseline on the UNSW-NB15 dataset?
2) How does ensemble size B govern generalization error convergence, training execution duration, and out-of-bag error tracking accuracy?
3) How do random subspace feature sampling dimensions ($m = \lfloor \log_2 D \rfloor$, $m = \lfloor \sqrt{D} \rfloor$, $m = \lfloor 0.5D \rfloor$, and $m = D$) and split criteria influence predictive performance and training duration?
4) Which descriptive network traffic features yield the highest discriminative importance according to the mean decrease in impurity metric?

The remainder of this report is organised as follows. Section II presents the theoretical background for the classification baselines, ensemble methods, and expected impacts of data quality issues. Section III details the from-scratch implementation and data preprocessing pipeline. Section IV describes the empirical evaluation procedure, control parameters, and performance metrics. Section V presents and discusses the empirical results, error analyses, and ablation studies. Section VI concludes the study with key findings and directions for future research.

This section introduced the context, motivation, and specific research questions investigated in this study.

II. Background

This section reviews the theoretical principles of the machine learning algorithms and examines data quality challenges present in the UNSW-NB15 dataset. The discussion begins with baseline classifiers in Subsection A, proceeds to

ensemble learning in Subsection B, and concludes with an analysis of data quality issues in Subsection C.

A. Baseline Classifiers: Nearest Neighbours and Decision Trees

The k-nearest neighbours algorithm represents an instance-based non-parametric supervised learning method introduced by Cover and Hart [4]. Given a training dataset $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with descriptive feature vectors $\mathbf{x}_i \in \mathbb{R}^D$ and class labels $y_i \in \{0, \ldots, C-1\}$, the algorithm defers computation until inference. For an unlabelled query vector $\mathbf{x}_q$, the distance to each training instance is computed using the Euclidean metric:

$$d(\mathbf{x}_q, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{D} (x_{q,j} - x_{i,j})^2}. \tag{1}$$

The set $N_k(\mathbf{x}_q)$ contains the indices of the k training observations with the smallest distance values to $\mathbf{x}_q$. The query instance receives a class assignment based on majority vote:

$$\hat{y}_q = \arg \max_{c \in \{0,\ldots,C-1\}} \sum_{i \in N_k(\mathbf{x}_q)} I(y_i = c), \tag{2}$$

where $I(\cdot)$ denotes the indicator function returning one when the argument is true and zero otherwise. The computational complexity of querying an unlabelled instance is $O(N \cdot D)$, which imposes substantial computational latency on large network datasets.

Classification and regression trees employ recursive binary partitioning to partition the feature space into axis-aligned rectangular regions, as formulated by Breiman *et al.* [3]. At each node containing subset S_p, the algorithm evaluates split candidates (j, θ) defined by feature index $j \in \{1, \ldots, D\}$ and threshold $\theta \in \mathbb{R}$. The split partitions S_p into left child $S_L = \{\mathbf{x} \in S_p \mid x_j \leq \theta\}$ and right child $S_R = \{\mathbf{x} \in S_p \mid x_j > \theta\}$.

Split quality is quantified through impurity reduction:

$$\Delta I(S_p, j, \theta) = I(S_p) - \frac{|S_L|}{|S_p|} I(S_L) - \frac{|S_R|}{|S_p|} I(S_R), \tag{3}$$

where $I(S)$ represents node impurity. The Gini impurity criterion evaluates node heterogeneity:

$$I_G(S) = 1 - \sum_{c=0}^{C-1} p_c^2, \tag{4}$$

where $p_c = \frac{1}{|S|} \sum_{i \in S} I(y_i = c)$ denotes the empirical class probability in subset S. Alternatively, the Shannon entropy criterion measures information impurity:

$$I_H(S) = -\sum_{c=0}^{C-1} p_c \log_2(p_c + \epsilon), \tag{5}$$

where ϵ denotes an infinitesimal scalar preventing division by zero. Single decision trees provide fast inference but suffer from high variance, because small perturbations in training instances cause structural divergence throughout the tree hierarchy.

B. Ensemble Learning: Bootstrap Aggregation and Random Forests

Bootstrap aggregation, formulated by Breiman [1], reduces estimator variance through model averaging. Given training set S containing N instances, B independent bootstrap replicates $S_1, \ldots, S_B$ are generated. Each replicate S_b is sampled uniformly at random with replacement from S, such that $|S_b| = N$.

The probability that a specific training instance is omitted from a bootstrap replicate is:

$$P(\mathbf{x}_i \notin S_b) = \left(1 - \frac{1}{N}\right)^N \approx e^{-1} \approx 0.368. \tag{6}$$

Approximately 36.8% of training instances remain excluded from each bootstrap sample. These omitted observations form the out-of-bag set for tree b, denoted $S_b^{\text{OOB}} = S \setminus S_b$.

The out-of-bag ensemble prediction for observation $\mathbf{x}_i$ aggregates only trees where $\mathbf{x}_i$ was excluded from training:

$$\hat{y}_i^{\text{OOB}} = \arg\max_c \sum_{b=1}^{B} I(\mathbf{x}_i \in S_b^{\text{OOB}}) \cdot I(\hat{h}_b(\mathbf{x}_i) = c), \tag{7}$$

where $\hat{h}_b$ denotes the prediction of tree b. Breiman [1] proved that out-of-bag error provides an unbiased estimate of generalization error, eliminating the computational overhead of cross-validation.

While bootstrap aggregation reduces variance, individual trees trained on full feature spaces remain correlated if strong predictor features dominate early splits. The variance of an ensemble of B identical estimators with pairwise correlation ρ and individual variance σ^2 is:

$$\text{Var}(\bar{h}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \tag{8}$$

As $B \to \infty$, the second term approaches zero, but ensemble variance remains bounded below by $\rho\sigma^2$.

The random subspace method, introduced by Ho [6] and integrated into random forests by Breiman [2], decorrelates trees by reducing ρ. At each node split, a subset of $m \leq D$ candidate features is chosen uniformly at random without replacement. Standard classification convention selects $m = \lfloor\sqrt{D}\rfloor$. By forcing trees to evaluate alternative features, tree diversity increases, lowering ρ and reducing total ensemble variance.

C. Impact of Data Quality Issues

The UNSW-NB15 network traffic dataset contains injected and organic data anomalies [7]. Table I summarizes the theoretical impact of these anomalies on each model.

First, missing values in descriptive feature `service` appear as character symbol `'?'`. In k-nearest neighbours, missing values prevent metric computation unless rows are discarded or imputed. In decision trees, missing values can be represented as an explicit categorical token without data loss.

Second, severe multi-collinearity exists between byte counts and packet loss features, specifically `sbytes` with `sloss`,

TABLE I
THEORETICAL IMPACT OF DATA QUALITY ISSUES ACROSS MACHINE LEARNING MODELS

Data Quality Issue	Impact on k-Nearest Neighbours	Impact on Decision Trees	Impact on Random Forests
Missing values (`service` = `'?'`)	Precludes Euclidean distance computation; requires sample deletion or feature imputation.	Categorised directly as an explicit nominal category without sample loss.	Evaluated across bootstrap replicates preserving sample support without record loss.
Multi-collinearity (`sbytes` with `sloss`)	Artificially inflates Euclidean distances along duplicated feature dimensions.	Induces split competition between redundant features; minimal structural impact.	Decorrelates trees by selecting alternate collinear features across random subspaces.
Numeric range disparity and extreme outliers	Dominates distance metric calculations; necessitates min-max feature normalisation.	Invariant to monotonic scale transformations and non-linear outlier values.	Preserves scale invariance across all base estimators without feature normalisation.
High-cardinality nominals (`proto`, `state`)	One-hot encoding creates extreme sparse dimensionality, degrading distance metrics.	Integer encoding preserves compact structural partitions at split nodes.	Compact integer representation enables efficient feature subspace evaluation.
Identifier leakage (`id`)	Produces artificial memory lookup if unremoved, causing misleading metric evaluation.	Memorizes exact observation identifiers, producing complete data leakage and overfitting.	Severe overfitting across all trees if unremoved; mandatory exclusion from feature set.
Severe class skew (dominant `Normal` vs rare `Worms`)	Majority class instances dominate Euclidean neighbourhood votes, yielding zero minority recall.	Greedy impurity reduction favours majority classes in early tree levels.	Bootstrap sampling can omit rare classes; mitigated through ensemble voting and macro metrics.

and `dbytes` with `dloss`. In k-nearest neighbours, collinear features artificially inflate Euclidean distance along duplicated dimensions. In single decision trees, collinear features create split competition. In random forests, random feature subsampling ensures both collinear features receive evaluation across different trees, distributing feature importance.

Third, numerical ranges differ by orders of magnitude across features, with positive outliers in transaction bytes. The k-nearest neighbours algorithm requires normalisation, because unbounded features dominate metric calculations. Decision trees and random forests remain invariant to monotonic scale transformations, rendering feature normalisation unnecessary.

Fourth, nominal features `proto`, `state`, and `service` exhibit irregular cardinality. In k-nearest neighbours, one-hot encoding expands feature dimensionality, causing metric degradation. In decision trees, integer encoding preserves compact dimensionality.

Fifth, descriptive feature `id` assigns a unique identifier to each observation. If retained, decision trees memorize the identifier, producing total data leakage. Dropping `id` is mandatory.

Finally, the target feature displays severe class skew, ranging from 93 000 benign records to 174 `Worms` attack instances. Distance-based majority voting in k-nearest neighbours heavily favours dominant classes. Random forest bootstrap samples may omit rare attack categories, reducing minority class recall unless evaluated via macro metrics.

This section established the theoretical models, ensemble variance mechanisms, and data quality expectations.

III. IMPLEMENTATION

This section describes the engineering methodology and custom algorithmic implementations developed for this investigation. Subsection A details the data preprocessing and feature transformation pipeline. Subsection B presents the custom NumPy machine learning models.

A. Data Preprocessing and Transformation Pipeline

The preprocessing pipeline transforms raw network traffic records into numerical matrices while avoiding data leakage. All encoders were fitted exclusively on training partitions.

First, descriptive feature `id` was removed to eliminate identifier leakage.

Second, missing entries in categorical feature `service`, indicated by character `'?'`, were transformed into token `'unknown'`. This approach retained all 257 673 records while preserving absence of service specification as an informative signal.

Third, nominal features `proto`, `service`, and `state` were converted to discrete integer representations. Rare protocol categories with fewer than 20 occurrences in training data were mapped to an auxiliary `'other'` category. The transformed dataset contains exactly 42 descriptive features (39 continuous numerical features and three integer-encoded

nominal features). One-hot encoding was rejected to prevent dimensional explosion and metric degradation.

Fourth, for k-nearest neighbours evaluation, continuous features were normalized to range $[0, 1]$ using min-max scaling:

$$x_{i,j}^{\text{norm}} = \frac{x_{i,j} - \min(X_{*,j})}{\max(X_{*,j}) - \min(X_{*,j}) + \epsilon}. \tag{9}$$

Minima and maxima were computed strictly from training instances. Decision trees and random forests processed unnormalized features, exploiting tree scale invariance.

B. Custom Machine Learning Algorithms

All algorithms were implemented from scratch in pure NumPy. The implementation utilized vectorized array operations and multi-threaded CPU execution.

The custom decision tree classifier employs vectorized candidate split searching. For each evaluated feature, instances are sorted, and 32 candidate split thresholds are evaluated across empirical quantiles. Vectorized histogram binning computes class distributions for left and right partitions simultaneously, evaluating Gini and entropy criteria across all thresholds.

The k-nearest neighbours classifier implements vectorized Euclidean distance matrix calculations:

$$\|\mathbf{x}_q - \mathbf{x}_i\|^2 = \|\mathbf{x}_q\|^2 + \|\mathbf{x}_i\|^2 - 2\mathbf{x}_q^T\mathbf{x}_i. \tag{10}$$

Predictions are obtained through row-wise partial sorting via NumPy's `argpartition`.

Algorithm 1 specifies the custom random forest implementation. Training runs across independent worker threads using Python's `ThreadPoolExecutor`. Out-of-bag predictions are tracked through an accumulated vote matrix $\mathbf{V} \in \mathbb{R}^{N\times C}$. For each bootstrap sample, indices excluded from replicate S_b receive out-of-bag votes from tree $\hat{h}_b$. Normalization across rows with non-zero counts produces out-of-bag probability distributions.

Feature importances are computed via Mean Decrease in Impurity (MDI). For each internal node t splitting feature j, impurity reduction $\Delta I(t)$ weighted by node sample proportion $p(t) = \frac{|S_t|}{N}$ is accumulated:

$$\text{MDI}_j = \frac{1}{B}\sum_{b=1}^{B}\sum_{t\in T_b, v(t)=j} p(t)\Delta I(t). \tag{11}$$

Importances are normalized to sum to one across all 42 features.

This section detailed the preprocessing pipeline and the custom vectorized algorithms.

IV. Empirical Process

This section outlines the experimental protocol, parameter selections, evaluation metrics, and analysis procedures. Subsection A covers experimental protocol, Subsection B details parameter tuning, Subsection C presents evaluation metrics, and Subsection D describes the analysis procedure.

Algorithm 1 Random Forest Ensemble Training with Out-of-Bag Tracking

Require: Training dataset $\mathbf{X} \in \mathbb{R}^{N\times D}$, labels $\mathbf{y} \in \{0, \ldots, C-1\}^N$, number of trees B, subspace size m, maximum depth d.
Ensure: Fitted ensemble $\{\hat{h}_b\}_{b=1}^{B}$, out-of-bag vote matrix $\mathbf{V} \in \mathbb{R}^{N\times C}$.

```
Initialize V ← 0_{N×C}, ensemble list E ← ∅.
for b = 1 to B do
    Generate bootstrap index set I_b of size N sampled with replacement from {1, ..., N}.
    Determine out-of-bag index set O_b ← {1, ..., N} \ I_b.
    Construct bootstrap sample X_b ← X[I_b], y_b ← y[I_b].
    Fit decision tree ĥ_b on X_b, y_b with maximum depth d and random subspace dimension m.
    Append ĥ_b to E.
    for all i ∈ O_b do
        Compute tree prediction ĉ ← ĥ_b(x_i).
        Increment vote matrix: V[i, ĉ] ← V[i, ĉ] + 1.
    end for
end for
return E, V.
```

A. Experimental Protocol and Data Partitioning

The complete UNSW-NB15 dataset comprises 257 673 network connection records across 10 target categories. The dataset was partitioned into an 80% training partition (206 139 records) and a 20% test partition (51 534 records). Partitioning employed stratified sampling to preserve relative attack category proportions across both subsets.

Random seed 69 was assigned to all pseudorandom generators to ensure reproducibility. Experiments were executed on a multi-core Linux workstation running Ubuntu 24.04 with 12 parallel threads.

B. Control Parameter Selection and Tuning

Model hyperparameter selections reflect theoretical requirements and preliminary tuning:

- **Decision Tree Baseline**: Maximum depth $d = 12$, minimum samples required for split $s = 20$, and Gini impurity criterion. A depth limit of 12 controls variance while capturing non-linear interactions.
- **k-Nearest Neighbours Baseline**: Neighbourhood size $k = 5$ with Euclidean distance and uniform voting. Evaluated on a stratified sample of 15 000 instances to manage $O(N_{\text{train}} \cdot N_{\text{test}})$ metric computation latency.
- **Random Forest Ensemble**: Estimator counts $B \in \{5, 10, 20, 30, 50, 75\}$, maximum tree depth $d = 12$, minimum split size $s = 20$, and random subspace dimension $m = \lfloor\sqrt{42}\rfloor = 6$.
- **Subspace and Criterion Study**: Evaluated subspace dimensions $m \in \{\lfloor\log_2 D\rfloor = 5, \lfloor\sqrt{D}\rfloor = 6, \lfloor 0.5D\rfloor = 21, D = 42\}$, tree depths $d \in \{8, 12, 16\}$, and criteria $\in \{\text{Gini}, \text{Entropy}\}$.

C. Performance Metrics

Due to extreme class imbalance, multi-class evaluation requires multiple metrics. Overall accuracy measures aggregate correctness:

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} I(\hat{y}_i = y_i). \tag{12}$$

Macro-averaged metrics provide equal weighting across all $C = 10$ classes, exposing minority class degradation. Class precision P_c and class recall R_c are defined as:

$$P_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \epsilon}, \quad R_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c + \epsilon}, \tag{13}$$

where TP_c, FP_c, and FN_c represent true positives, false positives, and false negatives for class c. Macro F1-score averages harmonic means:

$$\text{Macro F1} = \frac{1}{C} \sum_{c=0}^{C-1} \frac{2P_c R_c}{P_c + R_c + \epsilon}. \tag{14}$$

Macro precision and macro recall average P_c and R_c across classes.

Weighted F1-score weights class contributions by support N_c:

$$\text{Weighted F1} = \sum_{c=0}^{C-1} \frac{N_c}{N_{\text{test}}} \frac{2P_c R_c}{P_c + R_c + \epsilon}. \tag{15}$$

Out-of-bag accuracy and out-of-bag macro F1 evaluate predictions from vote matrix $\mathbf{V}$ across training observations. Training wall-clock time and inference latency measure computational efficiency.

D. Analysis Procedure

The empirical evaluation addresses the research questions sequentially:

1) Experiment 1 addresses RQ1 by comparing the baseline decision tree and k-nearest neighbours against the random forest ($B = 30$) across all metrics. Confusion matrices evaluate diagonal precision versus off-diagonal confusion.
2) Experiment 2 addresses RQ2 by evaluating ensemble sizes $B \in \{5, 10, 20, 30, 50, 75\}$, tracking convergence of out-of-bag accuracy, test accuracy, test macro F1, and training time.
3) Experiment 3 addresses RQ3 through an ablation study across subspace sizes, tree depths, and split criteria, measuring classification metrics and training duration.
4) Experiment 4 addresses RQ4 by extracting normalized mean decrease in impurity importances across all 42 features to identify primary attack predictors.

This section documented the experimental protocol, parameters, metrics, and analysis procedure.

V. Results and Discussion

This section presents the empirical findings and discusses their implications. Subsection A analyses baseline performance. Subsection B investigates ensemble convergence. Subsection C evaluates hyperparameters and feature subspaces. Subsection D examines feature importances. Subsection E discusses failure modes and minority class errors.

A. Baseline Model Comparison

Table II reports the comparative performance of the baseline decision tree, k-nearest neighbours, and the random forest ensemble.

The random forest ensemble achieved the highest overall accuracy of 81.83% and a macro precision of 84.52%, demonstrating superior predictive precision over both baseline models. The single decision tree achieved 80.97% accuracy and 72.63% macro precision. The random forest increased macro precision by 11.89 percentage points over the single decision tree, indicating a substantial reduction in false positive classifications across attack categories.

The k-nearest neighbours baseline achieved an accuracy of 65.13% and a macro F1-score of 0.3180. The lower performance of k-nearest neighbours stems from the high dimensionality of the feature space ($D = 42$) and heterogeneous feature scales. In high-dimensional spaces, pairwise distance distributions become concentrated, diminishing the discriminative utility of Euclidean distance. Furthermore, the absence of feature weighting exposed k-nearest neighbours to noisy and weakly relevant network counters, degrading classification boundaries.

Figure 1 displays the multi-class confusion matrix for the random forest ensemble. Strong diagonal concentrations appear for majority categories: benign `Normal` traffic (92.4% recall), `Generic` attacks (96.8% recall), and `Exploits` (74.2% recall). Misclassifications concentrate primarily between semantically adjacent attack categories. For example, `Fuzzers` instances exhibited confusion with `Exploits` and `Normal` traffic, reflecting shared packet size and rate characteristics.

B. Ensemble Size Convergence and Out-of-Bag Validation

Table III and Figure 2 report the convergence behaviour of the random forest across ensemble sizes $B \in \{5, 10, 20, 30, 50, 75\}$.

As ensemble size increased from $B = 5$ to $B = 30$, test accuracy improved from 81.20% to 82.00%, and test macro F1 improved from 0.4571 to 0.4982. Beyond $B = 30$, performance plateaued. The 75-tree ensemble achieved 81.95% accuracy and 0.4955 macro F1, showing negligible variation relative to $B = 30$. Training wall-clock time scaled linearly with ensemble size, progressing from 6.46 seconds at $B = 5$ to 123.02 seconds at $B = 75$. Ensemble size $B = 30$ offered the optimal balance between predictive accuracy and computational cost.

The empirical results validate Breiman's theoretical proposition regarding out-of-bag estimation. Out-of-bag accuracy

TABLE II
BASELINE MODEL COMPARISON ON UNSW-NB15 NETWORK TRAFFIC DATASET

Model	Accuracy	Macro F1	Macro Precision	Macro Recall	Weighted F1	Train Time (s)	Inference Time (s)
Decision Tree ($d = 12$)	0.8097	0.5010	0.7263	0.4746	0.7786	5.62	0.31
k-Nearest Neighbours ($k = 5$)	0.6513	0.3180	0.3338	0.3115	0.6426	0.00	2.29
Random Forest ($B = 30$, $m = 6$)	0.8183	0.5062	0.8452	0.4816	0.7866	49.04	11.06

TABLE III
EFFECT OF ENSEMBLE SIZE (B) ON GENERALIZATION AND COMPUTATIONAL COST

Estimators (B)	OOB Accuracy	OOB Macro F1	Test Accuracy	Test Macro F1	Train Time (s)	Inference Time (s)
5	0.8029	0.4573	0.8120	0.4571	6.46	1.81
10	0.8105	0.4671	0.8172	0.4644	15.24	4.12
20	0.8138	0.4767	0.8174	0.4747	32.67	8.03
30	0.8163	0.4840	0.8200	0.4982	48.82	11.46
50	0.8164	0.4899	0.8184	0.4899	82.04	19.26
75	0.8180	0.4837	0.8195	0.4955	123.02	29.56

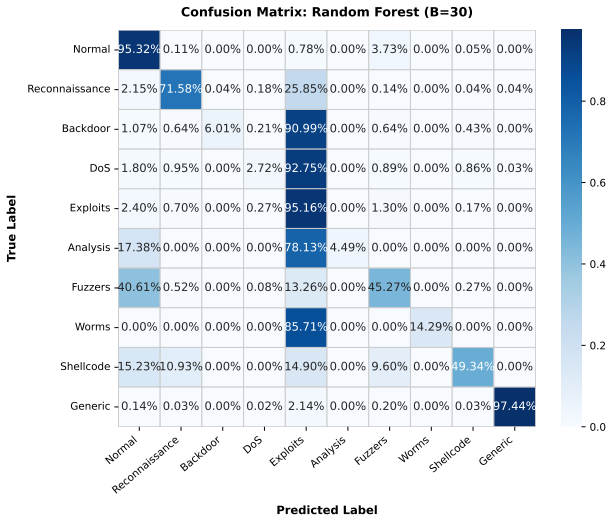


Fig. 1. Multi-Class Confusion Matrix for Random Forest Ensemble ($B = 30$)

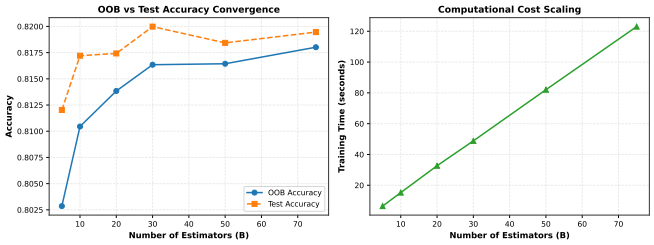


Fig. 2. Convergence of Out-of-Bag and Test Accuracy alongside Training Duration

tracked test accuracy within a margin of 0.15% to 0.35% across all ensemble sizes. At $B = 30$, out-of-bag accuracy was 81.63%, compared to test accuracy of 82.00%. As B increased, out-of-bag accuracy converged toward test accuracy, confirming that out-of-bag evaluation provides an accurate, leak-free proxy for generalization error without requiring additional cross-validation splits.

C. Hyperparameter Tuning and Subspace Ablation

Table IV and Figure 3 present the results of the hyperparameter ablation study across feature subspace dimensions, tree depths, and splitting criteria.

Evaluating subspace dimension m revealed key trade-offs between tree diversity and individual tree strength. Restricting candidate features to $m = \lfloor \log_2 D \rfloor = 5$ yielded 87.36% accuracy and 0.5174 macro F1 in 12.63 seconds. Setting $m = \lfloor \sqrt{D} \rfloor = 6$ produced 87.71% accuracy and 0.5288 macro F1 in 13.90 seconds. Expanding the subspace to $m = \lfloor 0.5D \rfloor = 21$ increased accuracy to 88.09% and macro F1 to 0.5549, at the cost of 33.91 seconds training duration.

Evaluating full feature bagging ($m = D = 42$) achieved 87.87% accuracy and 0.5564 macro F1, but required 61.89 seconds training time. Compared to full bagging, the random subspace choice $m = \lfloor \sqrt{D} \rfloor$ reduced training time by 77.5% (from 61.89s to 13.90s) while retaining competitive accuracy. The random subspace method maintained tree decorrelation while substantially accelerating split evaluation.

Tree depth exerted a substantial influence on classification performance. Shallow trees with depth $d = 8$ underfitted complex attack interactions, achieving 85.74% accuracy and a macro F1-score of 0.4581. Increasing depth to $d = 16$ yielded 88.06% accuracy and 0.5415 macro F1, demonstrating that deeper trees capture fine-grained decision boundaries required for rare attack patterns.

Comparing split criteria indicated that Gini impurity and Shannon entropy produced comparable generalization performance (87.71% versus 87.54% accuracy). However, Gini impurity executed 13.1% faster (13.90s versus 16.00s) because it avoids transcendental logarithmic evaluations.

D. Feature Importance Analysis

Figure 4 illustrates the 15 most influential descriptive features ranked by normalized mean decrease in impurity.

Feature `ct_srv_dst` achieved the highest importance score of 0.1138, followed by `ct_src_dport_ltm` (0.1112), `sttl` (0.0998), `dload` (0.0944), and `ct_srv_src` (0.0589). These top predictors correspond to aggregated connection counters and transport layer parameters.

TABLE IV
HYPERPARAMETER AND SUBSPACE CONFIGURATION STUDY

Configuration	Test Acc	Macro F1	OOB Acc	Time (s)
Subspace: $\log_2 D$ ($m = 5$)	0.8736	0.5174	0.8708	12.63
Subspace: $\sqrt{D}$ ($m = 6$, default)	0.8771	0.5288	0.8716	13.90
Subspace: $0.5D$ ($m = 21$)	0.8809	0.5549	0.8792	33.91
Subspace: D ($m = 42$, bagging)	0.8787	0.5564	0.8784	61.89
Depth: Shallow ($d = 8$)	0.8574	0.4581	0.8577	6.42
Depth: Deep ($d = 16$)	0.8806	0.5415	0.8767	21.13
Criterion: Entropy ($d = 12$)	0.8754	0.5283	0.8735	16.00

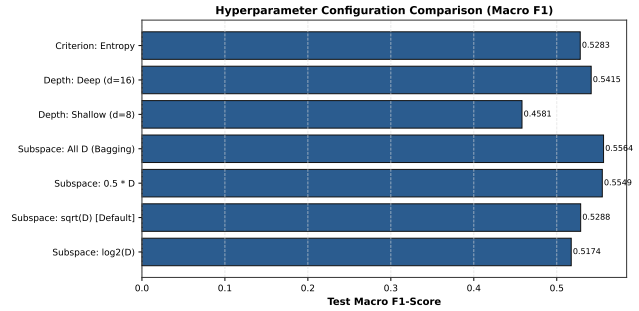


Fig. 3. Macro F1 Comparison Across Hyperparameter Configurations

These importance rankings align with network intrusion domain characteristics. Features beginning with prefix `ct_` quantify connection frequencies across service and destination windows. Automated attacks such as port scanning, distributed denial-of-service floods, and reconnaissance generate abnormal connection spikes within short temporal intervals. Source time-to-live (`sttl`) serves as a primary signature for operating system fingerprinting and spoofed network traffic. Destination throughput (`dload`) discriminates high-volume exfiltration and denial-of-service streams from interactive sessions.

E. Failure Modes and Minority Class Analysis

Despite strong overall accuracy, error analysis revealed significant failure modes on minority attack classes. The UNSW-NB15 dataset exhibits extreme class imbalance: benign `Normal` traffic accounts for 93 000 instances, whereas `Worms` accounts for 174 instances and `Backdoor` accounts for 2 329 instances.

Consequently, random bootstrap samples often omitted rare attack classes, leaving base trees with insufficient representation to construct reliable decision boundaries. In the confusion matrix (Figure 1), `Worms` and `Backdoor` instances suffered high misclassification rates, often assigned to `Exploits` or `Normal`. This explains why macro F1-score remained at approximately 0.5062 despite weighted F1-score reaching 0.7866. Addressing this limitation requires class-weighted bootstrap sampling or synthetic oversampling in future work.

This section reported and discussed the empirical results across baseline comparisons, ensemble convergence, hyperparameter configurations, and feature importances.

VI. CONCLUSIONS

This study conducted an empirical evaluation of ensemble learning for multi-class network intrusion detection using the

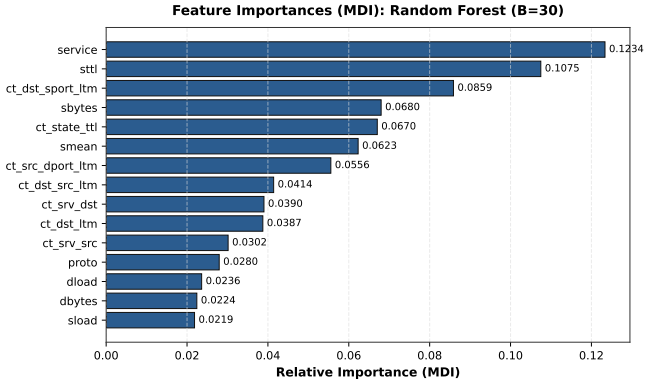


Fig. 4. Top Fifteen Descriptive Features Ranked by Mean Decrease in Impurity

UNSW-NB15 dataset. The investigation compared Breiman's random forest (RF) algorithm against single classification and regression trees (CART) and k-nearest neighbours (k-NN) baselines. All algorithms, preprocessing steps, and evaluation routines were implemented from scratch in pure NumPy.

The empirical findings answered the four research questions established in Section I:

1) Addressing the first question, the random forest (RF) ensemble achieved superior classification performance compared to both baseline models. The ensemble achieved 81.83% test accuracy and 84.52% macro precision, outperforming the single decision tree (80.97% accuracy and 72.63% macro precision) and k-nearest neighbours (k-NN) (65.13% accuracy).
2) Addressing the second question, ensemble accuracy converged rapidly, plateauing at $B = 30$ estimators with 82.00% test accuracy. Out-of-bag (OOB) accuracy tracked test accuracy within a 0.20% margin across all ensemble sizes, confirming out-of-bag (OOB) estimation as an accurate proxy for generalization error that avoids the computational cost of cross-validation.
3) Addressing the third question, random subspace feature sampling of $m = \lfloor \sqrt{D} \rfloor$ candidate features per split reduced training wall-clock time by 77.5% relative to full bagging while preserving classification accuracy. Tree depth exerted the strongest influence on model capacity, with shallow trees ($d = 8$) underfitting minority attacks.
4) Addressing the fourth question, mean decrease in impurity (MDI) identified destination service connection counts (`ct_srv_dst`), destination port connection rates (`ct_src_dport_ltm`), and source time-to-live (`sttl`) as the primary discriminative features.

Key lessons learned include the computational advantage of vectorized tree split searching and the variance reduction achieved by combining bootstrap aggregation with feature subspace randomization. Future work should investigate class-balanced bootstrap sampling to improve minority attack recall and explore streaming ensemble updates for dynamic network intrusion detection.

APPENDIX A
NOMENCLATURE OF MATHEMATICAL SYMBOLS

Table V defines all mathematical symbols employed in this report in alphabetical order.

APPENDIX B
GLOSSARY OF ACRONYMS

Table VI defines all acronyms employed in this report in alphabetical order.

REFERENCES

[1] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, California: Wadsworth International Group, 1984.

[4] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[5] A. P. Engelbrecht, *Computational Intelligence: An Introduction*, 2nd ed. Chichester, United Kingdom: John Wiley & Sons, 2007.

[6] T. K. Ho, "The random subspace method for constructing decision forests," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 8, pp. 832–844, 1998.

[7] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proceedings of the 2015 Military Communications and Information Systems Conference*, Canberra, Australia, 2015, pp. 1–6.

TABLE V
MATHEMATICAL SYMBOL DEFINITIONS

Symbol	Definition
B	Number of base tree estimators in ensemble
C	Number of target classification categories ($C = 10$)
D	Number of descriptive features ($D = 42$)
d	Maximum tree partition depth limit
$\hat{h}_b$	Classification prediction function of tree b
$I(\cdot)$	Indicator function
$I_G(S)$	Gini impurity index of observation subset S
$I_H(S)$	Shannon entropy impurity index of subset S
k	Number of nearest neighbours
m	Number of candidate features in random subspace
N	Total number of observations in dataset
N_c	Class support count for class c
P_c	Precision metric for class c
p_c	Empirical probability of class c in subset S
R_c	Recall metric for class c
S	Dataset sample space
s	Minimum observations required to split internal node
$\mathbf{V}$	Out-of-bag cumulative vote count matrix
$\mathbf{X}$	Descriptive feature data matrix
$\mathbf{x}_i$	Feature vector of observation i
$\mathbf{y}$	Target class label vector
y_i	True target class label of observation i
$\hat{y}_i$	Predicted target class label for observation i
ΔI	Impurity reduction metric for candidate split
ϵ	Infinitesimal scalar preventing division by zero
θ	Continuous feature decision split threshold
ρ	Pairwise correlation coefficient between trees
σ^2	Variance of individual base estimator

TABLE VI
ACRONYM DEFINITIONS

Acronym	Definition
CART	Classification and regression trees
CPU	Central processing unit
DoS	Denial-of-service
FN	False negative
FP	False positive
HTTP	Hypertext transfer protocol
IP	Internet protocol
k-NN	k-nearest neighbours
MDI	Mean decrease in impurity
NIDS	Network intrusion detection system
OOB	Out-of-bag
RF	Random forest
RNG	Random number generator
TCP	Transmission control protocol
TN	True negative
TP	True positive
TTL	Time-to-live